

B.Tech CSE Semester 5

Subject: System Design

Subject Code: TCS-504

ASSIGNMENT 1

Movie Ticket Booking System

<https://github.com/singhrohit92432-boop/movie-ticket-booking-system>

Name: Rohit Singh

Roll No: 2418864 / 39

Section: E2

SECTION A: REQUIREMENT ANALYSIS

Functional Requirements (FR1–FR8)

FR No.	Functional Requirement
FR1	The system shall display a list of all movies currently playing, including their title, language, and duration.
FR2	The system shall allow the customer to select a movie and view all available shows (screen number + start time) for that movie.
FR3	The system shall display the seat layout for a chosen show, marking each seat as AVAILABLE or BOOKED.
FR4	The customer shall be able to select one or more seats for a show. If any selected seat is already BOOKED, the entire booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds.
FR5	The system shall calculate the total price based on seat type: SILVER ₹150, GOLD ₹250, PLATINUM ₹400.
FR6	The system shall accept payment via exactly one method (UPI / Card / Cash) per booking. If payment fails, seats are released and booking status becomes FAILED.
FR7	The system shall print a ticket containing: booking ID, movie name, screen number, show time, seat numbers, and total amount.
FR8	The customer shall be able to cancel a booking. Upon cancellation, the seats become AVAILABLE again.

Non-Functional Requirements (NFR1–NFR4)

NFR No. Non-Functional Requirement

NFR1 Performance: The system must respond to user actions within 2 seconds, even

NFR No. Non-Functional Requirement

during peak usage.

NFR2 **Reliability:** Concurrent bookings for the same seat must be handled to prevent double-booking.

NFR3 **Usability:** The console menu must be intuitive with clear prompts and error messages.

NFR4 **Security:** Payment details (e.g., UPI ID, card number) must not be stored permanently or logged.

SECTION B: NOUN–VERB ANALYSIS

Noun Found	Keep as Class?	Reason
Movie	Yes	Has its own identity (title, duration, language) and independent data.
Seat	Yes	Has physical attributes (number, type, price) and state (booked/available).
Screen	Yes	Represents an auditorium with its own capacity and seat map.
Cinema	Yes	The theatre that contains multiple screens.
Show	Yes	A specific screening of a movie on a screen at a given time.
ShowSeat	Yes	Represents the status of ONE seat for ONE show (status varies per show).
Customer	Yes	Has identity (name, phone) and performs bookings.
Booking	Yes	Contains booking details: seats, total, status, time.
Payment	Yes	Abstract contract for paying via different methods.
"seat layout"	No	It's a view/print of Show's seats – not an object. Handled by a print method.
"ticket"	No	It's an output generated by TicketPrinter, not a stored entity.

SECTION C: CLASSES TABLE

Class	What it Knows (Attributes)	What it Does (Methods)	What it Must NOT Do
Movie	id, title, genre, duration, language, rating	getDetails(), getTitle(), getMovieId()	Must NOT handle seat selection or payment.
Screen	id, number, cinemaId, capacity, seatAvailability	getSeatMap(), checkAvailability(), bookSeat(), unbookSeat()	Must NOT create bookings or process payments.
Seat	id, row, number, type, isBooked	book(), unbook(), isAvailable(), getSeatInfo()	Must NOT calculate prices or handle payments.
Show	id, movieId, screenId, time, date	getAvailableSeats(), getShowId(), getMovieId()	Must NOT manage payments or users.
ShowSeat	id, showId, seatId, status (AVAILABLE/BOOKED)	book(), cancel(), getStatus()	Must NOT directly process payments.
Customer	id, name, phone, email	createBooking(), cancelBooking(), viewHistory()	Must NOT directly access database or payments.
Booking	id, customerId, seatIds, total, status, time	calculateTotal(), confirm(), cancel(), generateTicket()	Must NOT process payments directly.
Payment (abstract)	id, bookingId, amount, method, status, transactionId	processPayment() (pure virtual), refund(), verifyStatus()	Must NOT modify seat availability.
UPIPayment	upiId, provider	processPayment(), validateUPIId()	Must NOT handle Card or Cash payments.
CardPayment	cardNumber, holder, expiry, cvv	processPayment(), validateCard()	Must NOT handle UPI or Cash.
CashPayment	cashTendered,	processPayment(),	Must NOT handle

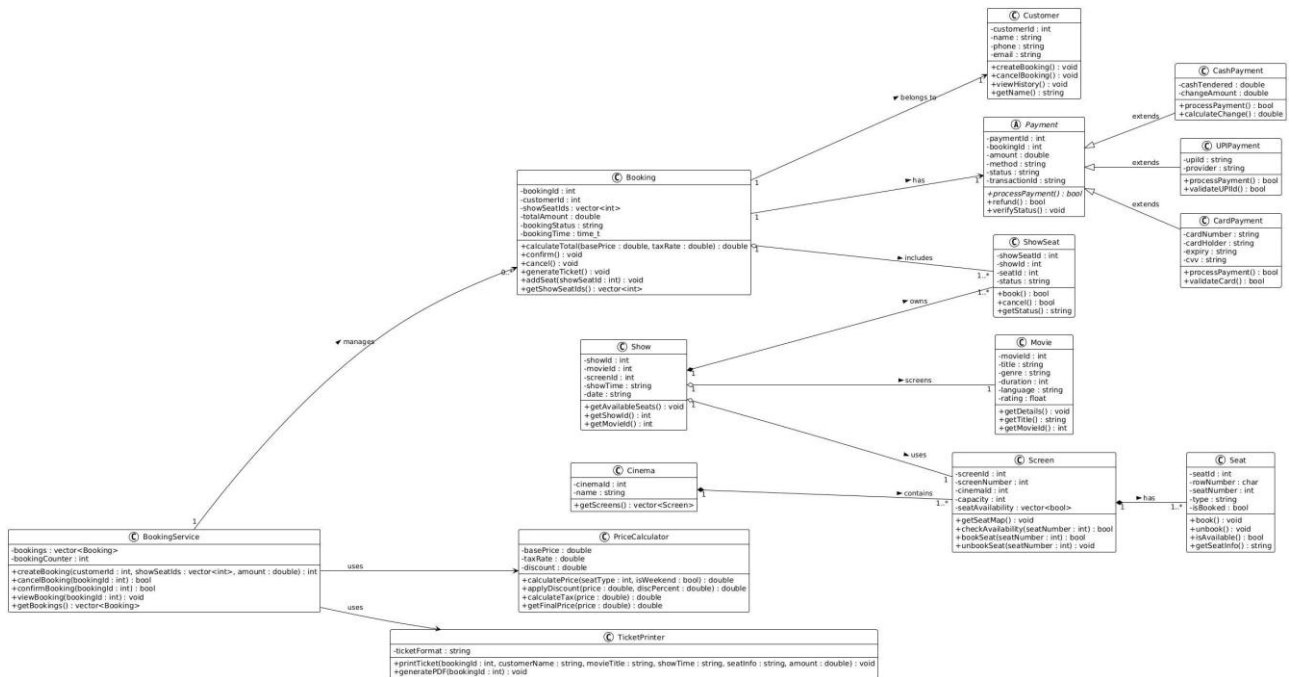
Class	What it Knows (Attributes)	What it Does (Methods)	What it Must NOT Do
PriceCalculator	changeAmount	calculateChange()	digital payments.
	basePrice, taxRate, discount	calculatePrice(), applyDiscount(), calculateTax(), getFinalPrice()	Must NOT modify bookings or seats.
TicketPrinter	ticketFormat	printTicket(), generatePDF()	Must NOT update booking status.
BookingService	bookings, bookingCounter	createBooking(), cancelBooking(), confirmBooking(), viewBooking()	Must NOT directly expose database operations.

SECTION D: RELATIONSHIPS

Pair	Type	Justification (Lifetime Test)
Cinema → Screen	Composition	If the cinema is destroyed, its screens are destroyed. Screens have no meaning without the cinema.
Screen → Seat	Composition	Seats exist only as part of a physical screen. If the screen is gone, the seats are gone.
Show → Movie	Aggregation	A movie exists independently of any show. If a show is cancelled, the movie still exists.
Show → Screen	Aggregation	A screen exists even if a show is cancelled. One screen hosts many shows over time.
Show → ShowSeat	Composition	ShowSeat is specific to one show. If the show is cancelled, the ShowSeat records (status for that show) are deleted.
Booking → Customer	Association	A customer can have many bookings. Booking and

Pair	Type	Justification (Lifetime Test)
		Customer exist independently.
Booking → ShowSeat	Aggregation	ShowSeat records exist independently of any booking. They can be linked/unlinked.
Booking → Payment	Association	Each booking has one payment. They exist together but are separate objects.
Payment → UPIPayment	Inheritance	UPIPayment IS-A Payment.
Payment → CardPayment	Inheritance	CardPayment IS-A Payment.
Payment → CashPayment	Inheritance	CashPayment IS-A Payment.
BookingService → Booking	Association	BookingService manages bookings but does not own them. Bookings exist independently.

SECTION E: CLASS DIAGRAM



The class diagram includes all 15 classes: Movie, Screen, Seat, Show, ShowSeat, Customer, Booking, Payment (abstract), UPIPayment, CardPayment, CashPayment, PriceCalculator, TicketPrinter, BookingService, and Cinema.

Composition (filled diamond): Cinema→Screen, Screen→Seat, Show→ShowSeat

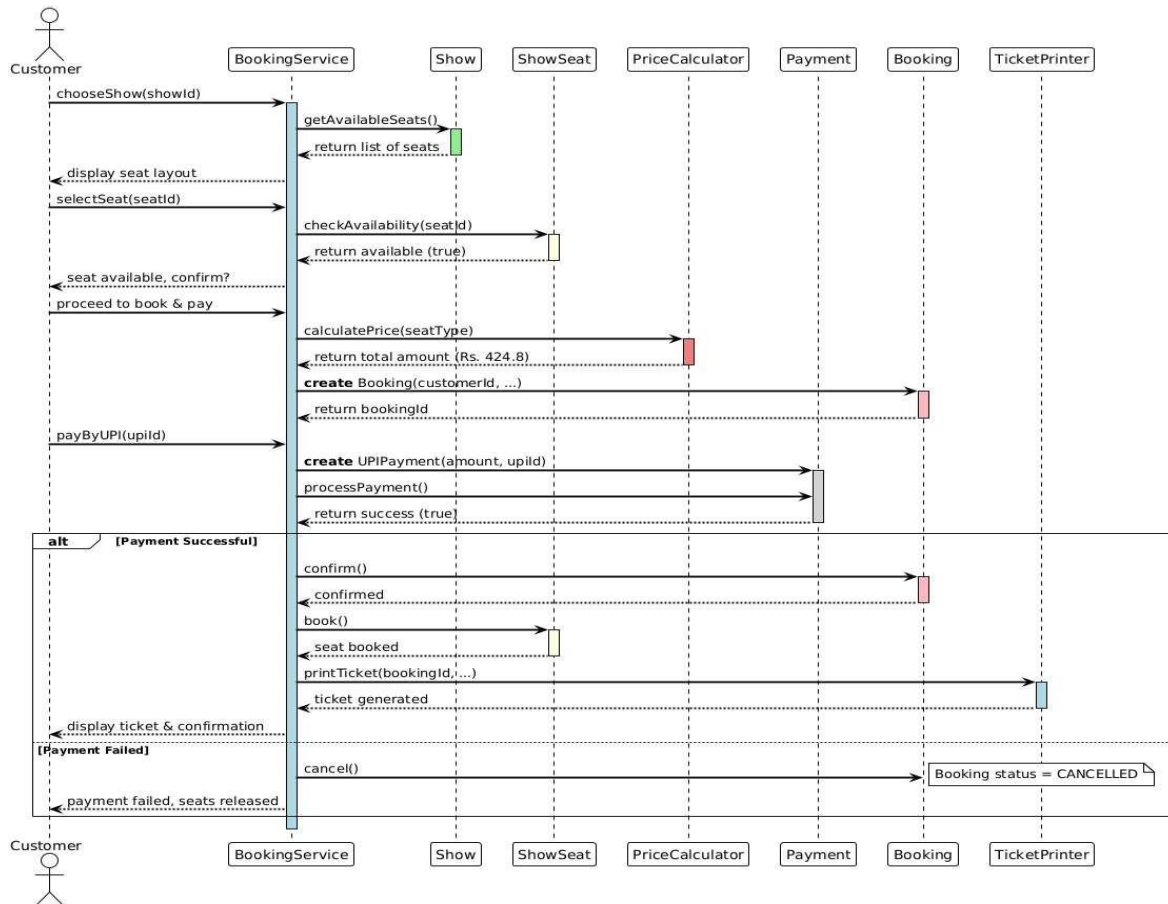
Aggregation (hollow diamond): Show→Movie, Show→Screen, Booking→ShowSeat

Inheritance (hollow triangle): Payment→UPIPayment/CardPayment/CashPayment

Association (arrow): Booking→Customer, Booking→Payment, BookingService→Booking

Multiplicities shown on both ends (e.g., 1, , 1..).

SECTION F: SEQUENCE DIAGRAM



Use case: Customer books 1 seat and pays by UPI.

Lifelines: Customer, BookingService, Show, ShowSeat, PriceCalculator, Payment, Booking, TicketPrinter.

Messages:

1. Customer → BookingService: chooseShow(showId)
2. BookingService → Show: getAvailableSeats()
3. Show → BookingService: return seat list
4. Customer → BookingService: selectSeat(seatId)
5. BookingService → ShowSeat: checkAvailability()
6. ShowSeat → BookingService: return available
7. Customer → BookingService: proceed to pay
8. BookingService → PriceCalculator: calculatePrice()
9. PriceCalculator → BookingService: return total

10. BookingService → Booking: «create» new Booking(...)
11. Customer → BookingService: payByUPI(upiId)
12. BookingService → Payment: «create» UPIPayment(...)
13. BookingService → Payment: processPayment()
14. Payment → BookingService: return success
15. BookingService → Booking: confirm()
16. BookingService → ShowSeat: book()
17. BookingService → TicketPrinter: printTicket()
18. TicketPrinter → BookingService: ticket printed
19. BookingService → Customer: display ticket & confirmation

SECTION G: MODULAR C++ CODE

File Structure

```
src/  
├── Movie.cpp  
├── Screen.cpp  
├── Seat.cpp  
├── Show.cpp  
├── ShowSeat.cpp  
├── Customer.cpp  
├── Booking.cpp  
├── Payment.cpp  
├── UPIPayment.cpp  
├── CardPayment.cpp  
├── CashPayment.cpp  
├── PriceCalculator.cpp  
├── TicketPrinter.cpp  
├── BookingService.cpp  
└── main.cpp
```

Demo Output (Screenshot)

```
movie-ticket-booking-...
  src
    CardPayment.cpp
    CashPayment.cpp
    Customer.cpp
    main
    main.cpp
    Movie.cpp
    Payment.cpp
    PriceCalculator.cpp
    Screen.cpp
    Seat.cpp
    Show.cpp
    ShowSeat.cpp
    TicketPrinter.cpp
    UPIPayment.cpp
    .gitignore
    README.md
    ticket-system

=====
MOVIE TICKET BOOKING SYSTEM
=====
1. List Movies    2. Book Ticket    3. Cancel Booking    4. View My Tickets    0. Exit
Enter your choice:1
Invalid choice. Try again.

=====
MOVIE TICKET BOOKING SYSTEM
=====
1. List Movies    2. Book Ticket    3. Cancel Booking    4. View My Tickets    0. Exit
Enter your choice: 2

===== BOOK A TICKET =====

===== CURRENT MOVIES =====
[1] 3 Idiots (Hindi) 170 min
[2] Interstellar (English) 169 min
Choose movie number: 2

===== SHOWS FOR Interstellar =====
[1] Screen-1 at 07:00 PM (Date: 2026-09-10)
Choose show number: 1
```

```
movie-ticket-booking-...
  src
    CardPayment.cpp
    CashPayment.cpp
    Customer.cpp
    main
    main.cpp
    Movie.cpp
    Payment.cpp
    PriceCalculator.cpp
    Screen.cpp
    Seat.cpp
    Show.cpp
    ShowSeat.cpp
    TicketPrinter.cpp
    UPIPayment.cpp
    .gitignore
    README.md

===== SEAT LAYOUT ( [ ] = AVAILABLE, [X] = BOOKED ) =====
      1    2    3    4
A   [ ] [ ] [ ] [ ]
B   [ ] [ ] [ ] [ ]
C   [ ] [ ] [ ] [ ]
Legend: Platinum (Row A), Gold (Row B), Silver (Row C)
Enter seat numbers (e.g., A1,B2,C3): A1
Total amount: Rs. 400
Pay by: 1. UPI    2. Card    3. Cash
1
Enter UPI ID (e.g., user@paytm): rohit@paytm
Processing UPI payment...
UPI ID: rohit@paytm
Provider: Paytm
Amount: Rs. 400
UPI payment successful!
Transaction ID: UPI-5000-1788540649
Enter your name: paytm
Enter phone number: 9086576834
Booking created with ID: 1001
Booking #1001 confirmed!
```

```
movie-ticket-booking-...
└─ src
   ├── CarPayment.cpp
   ├── CashPayment.cpp
   ├── Customer.cpp
   └─ main
      ├── main.cpp
      ├── Movie.cpp
      ├── Payment.cpp
      ├── PriceCalculator.cpp
      ├── Screen.cpp
      ├── Seat.cpp
      ├── Show.cpp
      ├── ShowSeat.cpp
      ├── TicketPrinter.cpp
      └── UPIPayment.cpp
.gitignore
README.md
ticket-system
```

```
Booking created with ID: 1001
Booking #1001 confirmed!

MOVIE TICKET

Booking ID : 1001
Customer   : paytm
Movie      : Interstellar
Show Time  : 07:00 PM
Seats      : A1
Total Amount : Rs. 400
Format     : PDF
Status     : CONFIRMED

Booking completed successfully!

=====
MOVIE TICKET BOOKING SYSTEM
=====
1. List Movies   2. Book Ticket   3. Cancel Booking   4. View My Tickets   0. Exit
Enter your choice: 0
Thank you for using the system!
```

SECTION H: SOLID PRINCIPLES

1. Single Responsibility Principle (SRP)

Principle: A class should have only one reason to change.

Implementation:

- PriceCalculator handles **only** pricing logic.
- TicketPrinter handles **only** ticket formatting and printing.
- Booking handles **only** booking state and operations.
- Payment handles **only** payment processing.

2. Open/Closed Principle (OCP)

Principle: Classes should be open for extension but closed for modification.

Implementation:

- Payment is an abstract base class.
- New payment methods (e.g., WalletPayment) can be added by creating a new subclass without modifying existing UPIPayment, CardPayment, or CashPayment.

3. Liskov Substitution Principle (LSP)

Principle: Subtypes must be substitutable for their base types.

Implementation:

- All payment subclasses (UPIPayment, CardPayment, CashPayment) can be used wherever Payment* is expected.
- Each subclass correctly implements processPayment() without requiring extra setup.